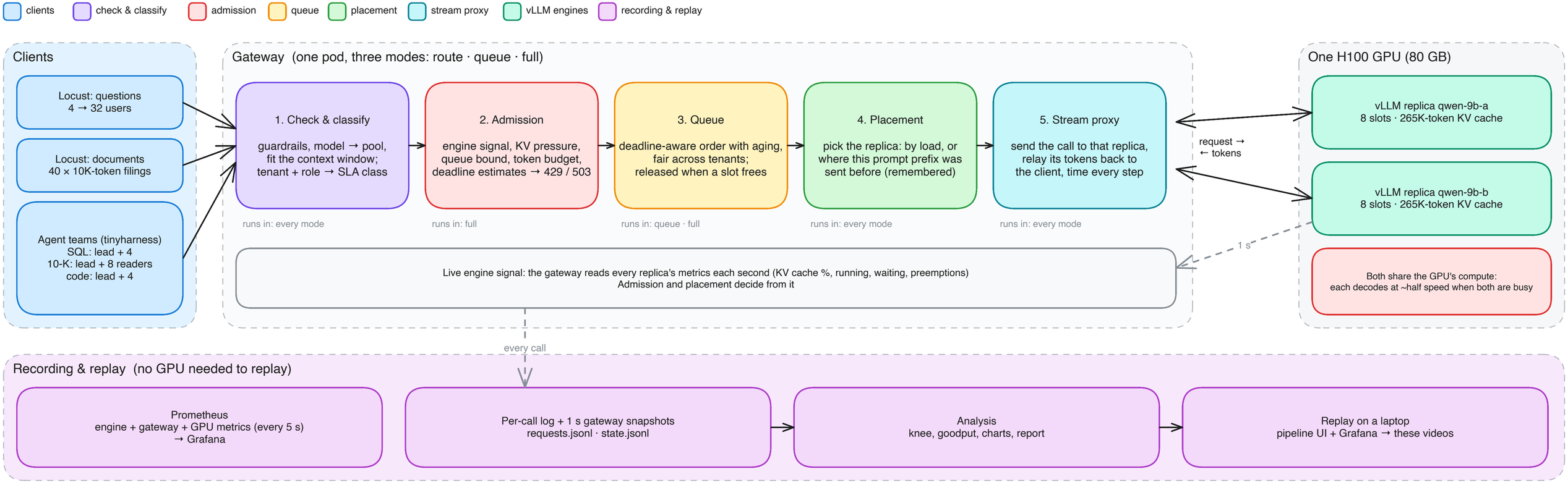

clients
check & classify
admission
queue
placement
stream proxy
vLLM engines
recording & replay
Clients
Locust: questions
4 → 32 users
Locust: documents
40 × 10K-token filings
Agent teams (tinyharness)
SQL: lead + 4
10-K: lead + 8 readers
code: lead + 4
Gateway (one pod, three modes: route · queue · full)
1. Check & classify
guardrails, model → pool,
fit the context window;
tenant + role → SLA class
runs in: every mode
2. Admission
engine signal, KV pressure,
queue bound, token budget,
deadline estimates → 429 / 503
runs in: full
3. Queue
deadline-aware order with aging,
fair across tenants;
released when a slot frees
runs in: queue · full
4. Placement
pick the replica: by load, or
where this prompt prefix was
sent before (remembered)
runs in: every mode
5. Stream proxy
send the call to that replica,
relay its tokens back to
the client, time every step
runs in: every mode
Live engine signal: the gateway reads every replica's metrics each second (KV cache %, running, waiting, preemptions)
Admission and placement decide from it
request →
← tokens
1 s
One H100 GPU (80 GB)
vLLM replica qwen-9b-a
8 slots · 265K-token KV cache
vLLM replica qwen-9b-b
8 slots · 265K-token KV cache
Both share the GPU's compute:
each decodes at ~half speed when both are busy
every call
Recording & replay (no GPU needed to replay)
Prometheus
engine + gateway + GPU metrics (every 5 s)
→ Grafana
Per-call log + 1 s gateway snapshots
requests.jsonl · state.jsonl
Analysis
knee, goodput, charts, report
Replay on a laptop
pipeline UI + Grafana → these videos